



Automated E-Commerce Deployment Platform

CloudMart

Automated E-Commerce Deployment Platform Overview

Implementing a DevOps-driven approach to seamlessly deploy and manage multi-service e-commerce platforms.



Problem Statement: E-Commerce Downtime & Complex Deployments

Traditional deployments face significant challenges:

- ✓ **Manual Setups**
Time-consuming and error-prone.
- ✓ **Server Drift**
Inconsistencies between environments over time.
- ✓ **Lack of Containerization**
"It works on my machine" issues.
- ✓ **Downtime Risks**
High probability of service interruption during updates.



Project Objectives: Achieving High Availability & Scalability

Key Technical Goals

- ✓ **Automate Deployment Pipelines**
Streamline the path from code to production.
- ✓ **Maintain High Uptime**
Ensure continuous service availability.
- ✓ **Robust Monitoring**
Implement real-time system observability.
- ✓ **Auto-Scaling**
Dynamically adjust resources under load.
- ✓ **Secure Rollbacks**
Enable quick recovery from failed deployments.



Comprehensive System Architecture & Technology Stack

High-Level Overview

Application Stack

Web, Database, Payment, Search

DevOps Toolchain

CI/CD & Cloud: Jenkins, Ansible, Terraform, AWS
Containers & Web: Docker, Kubernetes, Nginx, Git

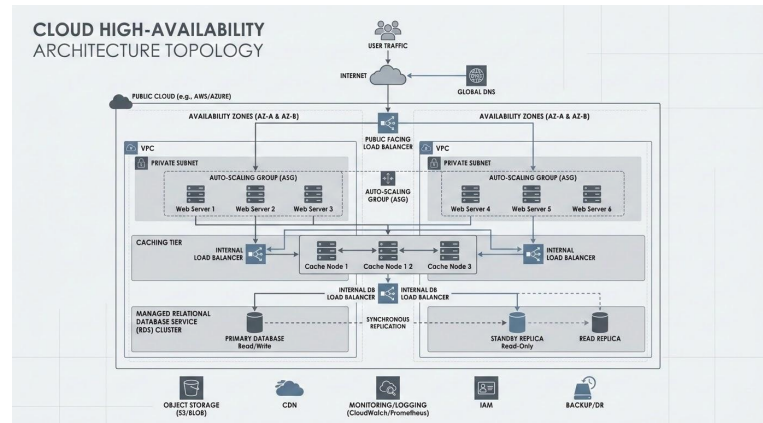
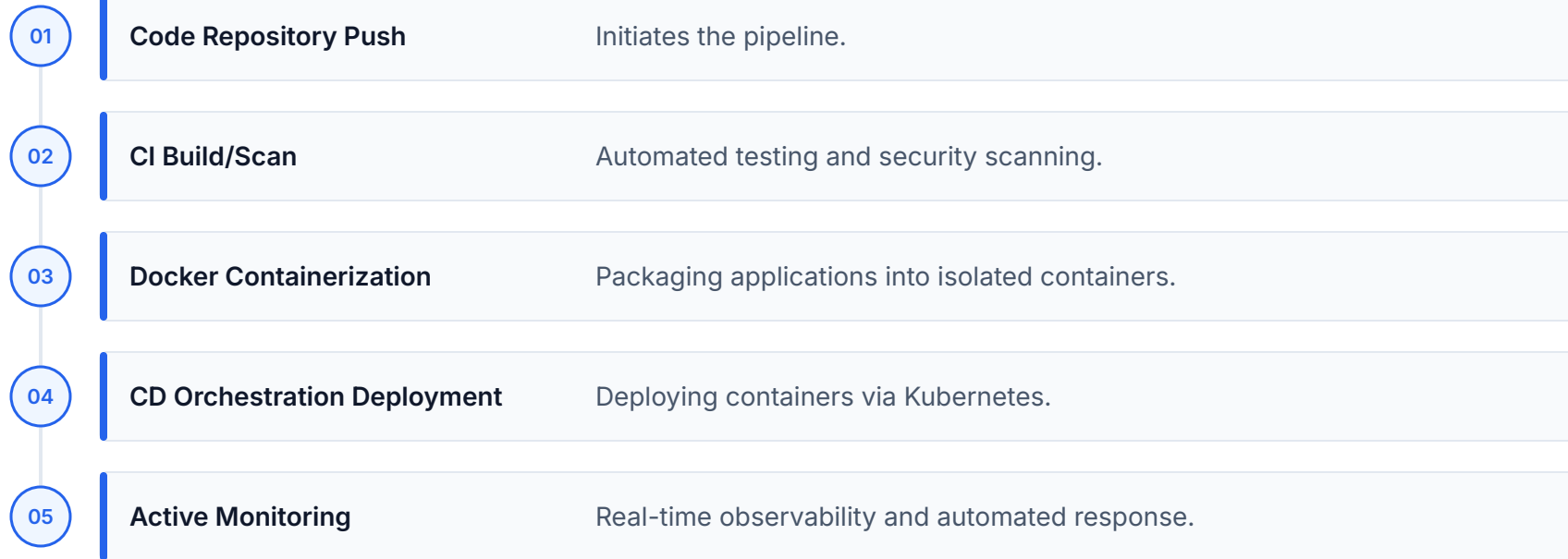


Figure 1: Cloud High-Availability Architecture Topology

Core DevOps Lifecycle



CI/CD, Containerization, and Provisioning





Infrastructure as Code (IaC)

AWS Provisioning with Terraform — Establishing the Environment Baseline



Declarative Setup

Defines the desired state of the AWS infrastructure.



Repeatable Provisioning

Ensures consistent environments across dev, staging, and prod.



Prevents Configuration Drift

Maintains a single source of truth.



Quick Replication

Enables rapid environment recreation.



Terraform Architecture

AWS EC2, RDS, S3, & ELB Design

EC2

AWS EC2

Provisioned for Kubernetes worker nodes.

RDS

AWS RDS

Secure database clusters for persistent data.

S3

AWS S3

Storage for static assets.

ELB

AWS ELB

Elastic Load Balancing for secure external traffic routing.



AWS Cloud Infrastructure Deployment Environment

Server Configuration Management: Automated Ansible Playbooks



Automating Host Setup

01 / PROVISION

Host Configuration

Automated setup of base server environments.

02 / ENVIRONMENT

System Dependencies

Managing required packages and libraries.

03 / SECURITY

Security Patches

Deploying necessary updates and hardening.

04 / RUNTIME

Container Runtimes

Configuring the environment for Docker/Kubernetes.



Microservices Architecture

Multi-Service Platform Overview

Decoupled Services



Web Frontend

The user-facing application interface.
[UI LAYER]



Database Layer

Persistent storage for application data.
[STORAGE]



Payment Gateway

Simulator for transaction processing.
[GATEWAY]



Search Service

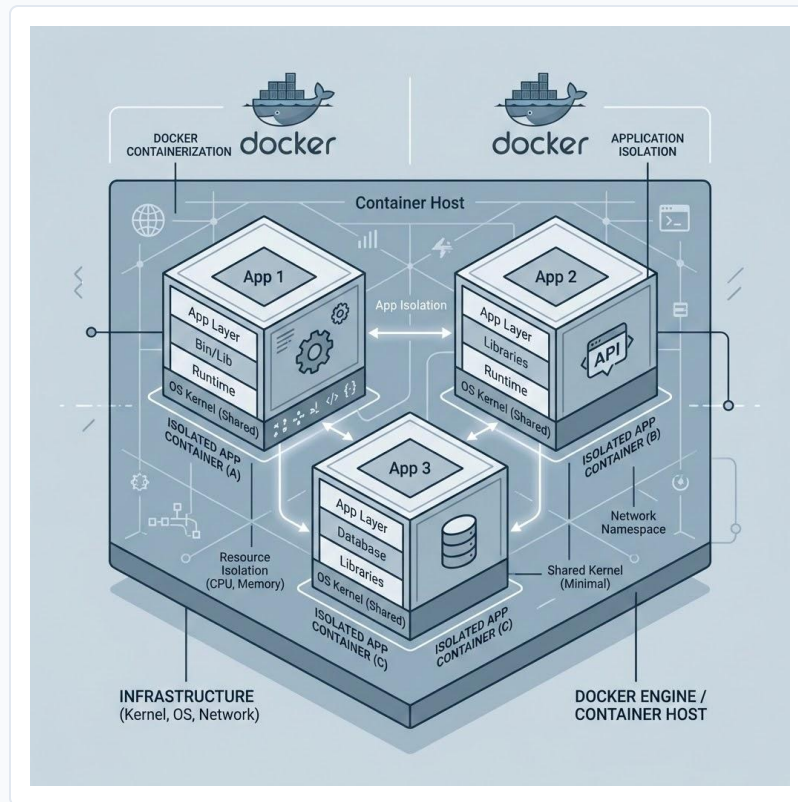
Elasticsearch integration for product discovery.
[SEARCH]



Dockerizing E-Commerce Services

Benefits of Containerization:

- ✓ **Application Isolation**
Services run independently without interference.
- ✓ **Environment Parity**
Consistency between development and production.
- ✓ **Fast Boot Times**
Lightning-fast startup compared to VMs.
- ✓ **Eliminates Discrepancies**
Solves the "it works on my machine" problem.





Build Optimization

Docker Build Configuration

Microservices Dockerfiles

- ✓ **Multi-Stage Builds**
Optimizing the build process for efficiency.
- ✓ **Minimized Image Sizes**
Reducing footprint for faster deployments.
- ✓ **Security**
Running containers as non-root users.
- ✓ **Dependency Caching**
Speeding up subsequent builds.

```
Dockerfile.microservice

# --- STAGE 1: Dependency Cache ---
FROM golang:1.21-alpine AS builder
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 go build -o app

# --- STAGE 2: Secure Runtime ---
FROM alpine:3.18
RUN adduser -D -u 10001 appuser
COPY --from=builder /src/app /app
USER appuser
EXPOSE 8080
CMD ["/app"]
```



Orchestration Layer: Kubernetes Deployment Topology

Managing Containers with Kubernetes

01 / COMPUTE

Pods

The smallest deployable unit one running container. CloudMart runs 14 pods total: 7 microservices, the frontend, and 6 databases.

02 / NETWORK

Services

A stable internal name that load-balances traffic to healthy pods. Any pod can reach another this way by name, never by IP, since pod IPs change every time one restarts.

03 / GATEWAY

Ingress

The one external entry point. A single Ingress routes by URL path /api/ to the gateway, everything else to the frontend.

04 / STATE

Deployments

Keeps the declared number of pod replicas alive, restarting any that crash. All 14 of CloudMart's workloads from api-gateway to order-db are managed this way.



Dynamic Scaling

Kubernetes Horizontal Pod Auto-scaler (HPA)

Automated Traffic Handling

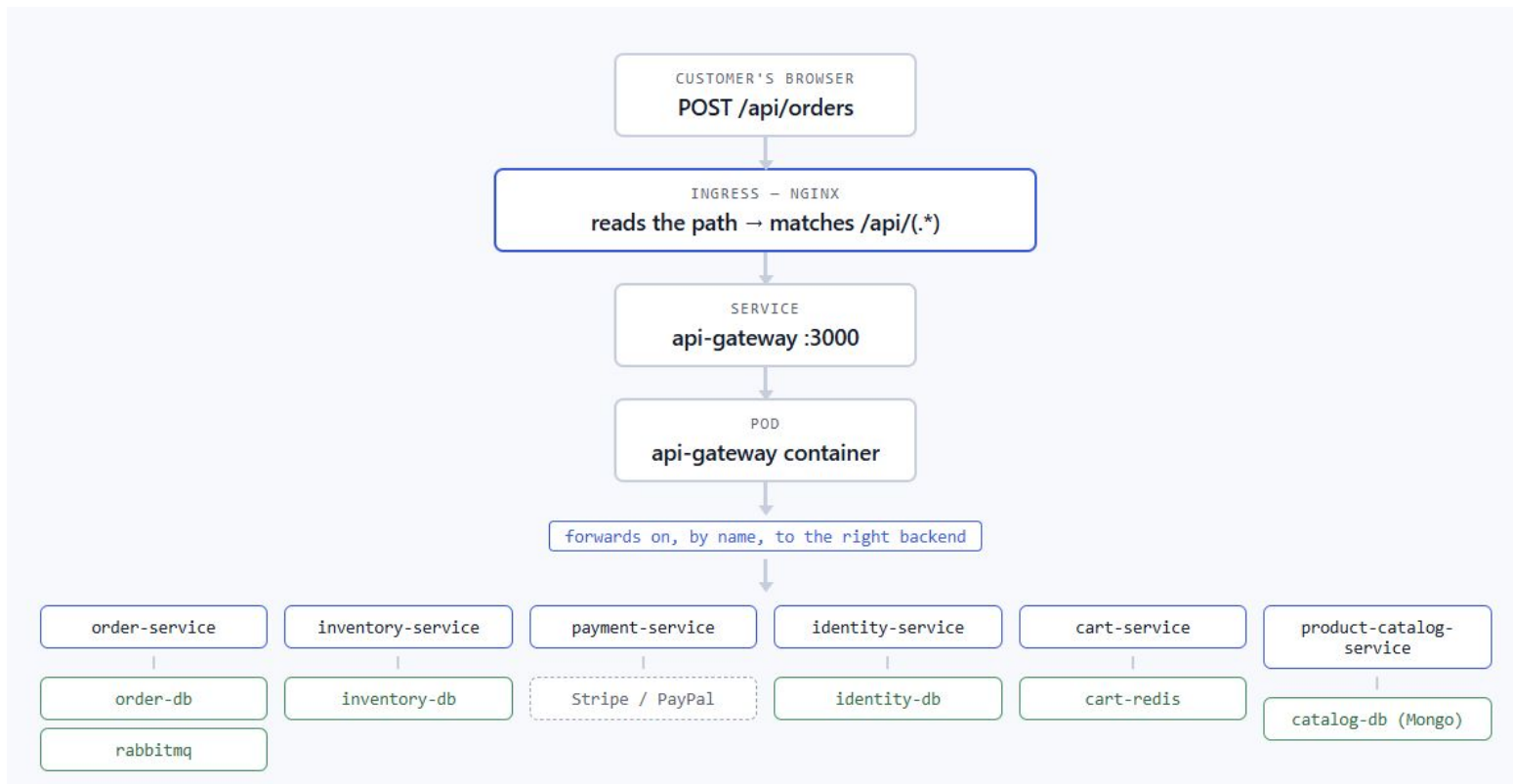
- ✓ **Traffic Spikes**
Scales order-service from 1 to 6 pods, Handled automatically without human intervention.
- ✓ **Thresholds**
Utilizes CPU and memory utilization thresholds, Triggers past 70% CPU or 80% memory usage.
- ✓ **HPA Configuration**
Scales pods based on defined metrics, Each service sets its own min/max
- ✓ **Scale-Down Safety**
Waits ~5 minutes of low usage before removing pods.

Warning: Ensure cluster resource request thresholds are configured carefully. Setting overly-aggressive limits without cool-down safeguards can result in rapid scheduling thrashing under sudden load-spike patterns.



Reverse Proxy & Routing

Routing Control via Nginx — Efficient Traffic Management





Continuous Integration Setup: Automation with Jenkins

Central Orchestration Engine



Webhooks Integration

Automated triggers from Git repositories.



Architecture

Master-agent setup for distributed builds.



Credentials Management

Secure handling of secrets and keys.



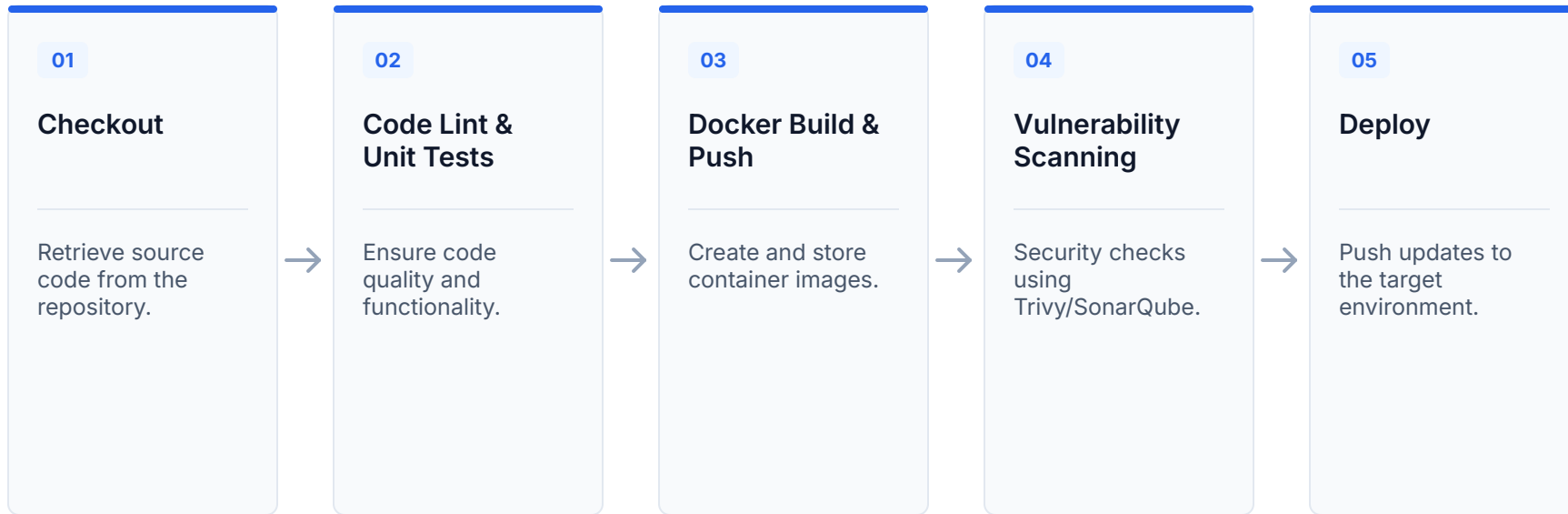
Pipeline-as-Code

Defined using Jenkinsfile for version control.

Automated Pipeline Workflow



Build, Test, and Scan Stages





Source Control Strategy

Structured Git Repository Setup

Organizing the Codebase

- ✓ **Structure:** Monorepo or polyrepo design.
- ✓ **Microservices Code:** Dedicated folders for application logic.
- ✓ **Infrastructure Files:** Terraform configurations.
- ✓ **Automation Scripts:** Ansible playbooks.
- ✓ **Orchestration:** Kubernetes manifests.

```
git-repository-tree

# Structured Repository Tree
project-monorepo/
├── apps/ # Microservices code
│   ├── api-service/
│   └── auth-service/
├── terraform/ # Infrastructure files
│   ├── main.tf
│   └── variables.tf
├── ansible/ # Automation scripts
│   └── playbooks/
├── k8s/ # Orchestration
└── manifests/
```



High Availability: Load Balancing & Resilient Traffic Distribution

Eliminating Single Points of Failure



01

AWS ELB

Elastic Load Balancing distributes incoming traffic.



02

Nginx Integration

Works alongside ELB and K8s services.



03

Kubernetes Services

Internal traffic routing within the cluster.



04

Multi-AZ Deployment

Distributes loads evenly across multiple availability zones.

Reliability Patterns: Automated Rollback Strategies



Handling Deployment Failures

Pattern 01

Automated Rollbacks

Implemented in Jenkins and Kubernetes.

Pattern 02

Health Endpoints

Continuous checking of service health.

Trigger Rule (Auto-Revert): Automatically reverts to the last known stable replica upon failure detection.



Monitoring Architecture: Service Metrics with Prometheus

Prometheus Architecture

01

Core monitoring framework.

Pull-Based Collection

02

Gathers metrics from configured endpoints.

Exporters

03

Configuring exporters for various services.

Custom Metrics

04

Defining application-specific telemetry.

System Observability



Prometheus Dashboard Metrics

System KPI

gauge

CPU Utilization

Tracking processing load.

System KPI

gauge

Memory Pressure

Monitoring memory usage.

Network KPI

histogram

HTTP Request Latency

Measuring response times.

Network KPI

counter

Error Rates

Tracking HTTP 5xx errors.

Database KPI

gauge

Database Connections

Monitoring active connections.



Visual Telemetry Presentation

Interactive Grafana Metrics Dashboards

- ✓ **Grafana Dashboards**
Designed to visualize Prometheus data.
- ✓ **Interactive Metrics**
Providing actionable insights to operators.
- ✓ **Alert Rules**
Configuring thresholds for system anomalies.
- ✓ **Integrations**
Sending alerts via Slack or Email.



Figure 1: Production Grafana telemetry monitor interface.

Security Hardening

IAM, Secrets Management, and Encryption



Protecting Infrastructure & Credentials

AWS SECURITY

AWS IAM

Utilizing roles with least privilege access.

DATA PROTECTION

Encryption

Encrypting static S3 buckets.

ORCHESTRATION

Kubernetes Secrets

Secure management of sensitive data.

VULNERABILITY MGT

Prevention

Preventing the leakage of database passwords and credentials.

Platform Evaluation



Core Deliverables Checklist

| Completed Package Review



Dockerfiles



CI/CD Pipelines



Kubernetes Configs



Ansible Scripts



Prometheus Monitoring



Nginx Proxy



Terraform Files



Structured Git Repository

Conclusion & Technical Capabilities Summary



DevOps Deployment System Success



Near-Zero Downtime

Achieved through automated strategies.



Reproducible Deployments

Highly consistent environments.



Robust Self-Healing

Automated recovery mechanisms.



Modern Pipeline

A fully modern DevOps pipeline ready for production.